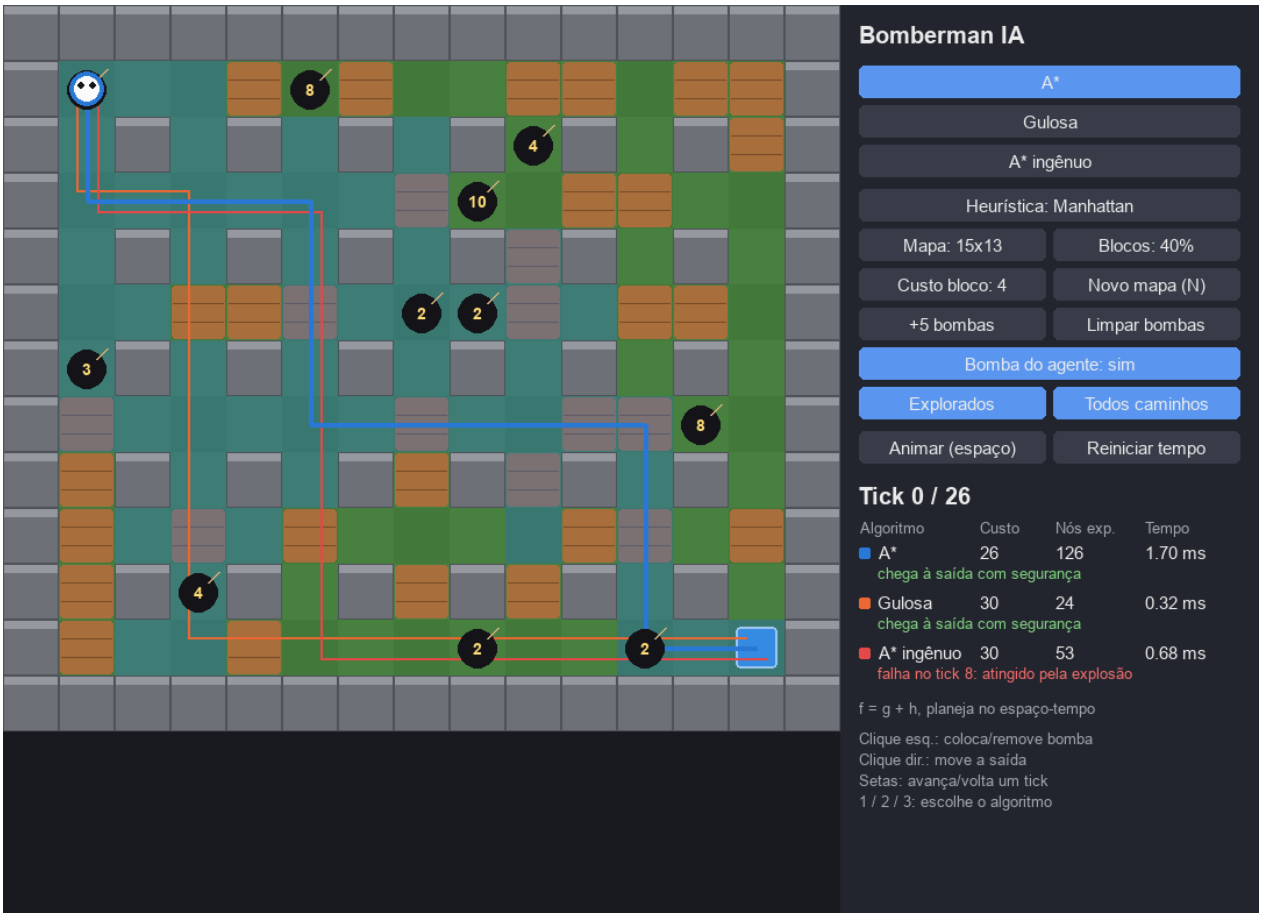
Bomberman IA

Guia de estudo do projeto

Como um agente de Bomberman encontra o caminho mais rápido e seguro até a saída usando **A*** e **Busca Gulosa**: os conceitos, a modelagem, o código e os resultados.



Sumário

1. Visão geral 4

O que o projeto faz . . . 4

Como rodar . . . 4

Controles da interface . . . 4

Como estudar . . . 5

2. O jogo e o problema 6

O Bomberman em 30 segundos . . . 6

O que o agente pode fazer . . . 6

Por que não é um labirinto comum . . . 6

3. Busca do zero 8

Os ingredientes de um problema de busca . . . 8

Como uma busca funciona . . . 8

Busca Gulosa: f(n) = h(n) . . . 8

A*: f(n) = g(n) + h(n) . . . 9

Heurística admissível: a condição para o A* ser ótimo . . . 9

Exemplo 1: Gulosa e A* no mesmo mapa . . . 9

4. Como o Bomberman vira um problema de busca 12

Os ingredientes, agora com tempo . . . 12

Restrições: os estados em que o agente morreria não existem . . . 12

O cronograma de explosões . . . 13

O horizonte: por que o espaço de estados não é infinito . . . 14

As heurísticas e por que elas funcionam . . . 14

Exemplo 2: esperar o fogo passar . . . 14

O A* ingênuo . . . 15

5. Tour pelo código 16

Organização das pastas . . . 16

O que acontece quando você clica no mapa . . . 16

ia/busca.py: o coração do projeto . . . 17

Os outros arquivos, em resumo . . . 19

Os testes automatizados . . . 19

6. Resultados 21

Cenário 1: tamanho do mapa . . . 21
Cenário 2: heurísticas . . . 22
Cenário 3: densidade de blocos . . . 22
Cenário 4: fuga das explosões . . . 23
7. Preparação para a apresentação 24
Roteiro sugerido para a demonstração . . . 24
Perguntas prováveis e respostas . . . 24
Limitações . . . 25
Possíveis melhorias . . . 25
8. Experimentos para fazer sozinho 26
Referências . . . 26

1. Visão geral

Este guia explica o projeto do zero, na ordem em que vale a pena estudar. Cada capítulo supõe que você leu os anteriores. Não é preciso saber nada de busca antes: o capítulo 3 começa pelo básico.

O que o projeto faz

Um agente de Bomberman precisa sair do canto superior esquerdo e chegar à **saída** no canto inferior direito, no **menor tempo possível e sem morrer**. No caminho há paredes, blocos destrutíveis e bombas prestes a explodir. O caminho é planejado por dois algoritmos de busca informada, o **A*** e a **Busca Gulosa**, que o projeto compara lado a lado. Um terceiro planejador, o **A* ingênuo**, ignora as bombas e serve para mostrar o que dá errado quando o tempo não é considerado.

Como rodar

Depois de clonar o repositório, crie o ambiente virtual e instale as dependências uma vez:

```
python3 -m venv .venv
.venv/bin/pip install -r requirements.txt
```

Comando	O que faz
`.venv/bin/python main.py`	abre o jogo (interface visual)
`.venv/bin/python -m exemplos.passo_a_passo`	imprime, nó por nó, o que cada algoritmo expande em mapas pequenos
`.venv/bin/python -m benchmark.executar`	roda os 4 cenários de teste e gera tabelas e gráficos em resultados/
`.venv/bin/python -m pytest testes`	roda os testes automatizados

Controles da interface

Ação	Como
Escolher o algoritmo	botões no painel ou teclas 1 (A*), 2 (Gulosa), 3 (A* ingênuo)
Colocar ou remover bomba	clique esquerdo numa célula do mapa
Mudar a saída de lugar	clique direito numa célula do mapa
Animar / pausar	barra de espaço ou botão Animar
Avançar / voltar um tick	setas → e ←
Novo mapa	tecla N ou botão Novo mapa
Heurística, tamanho, % de blocos, custo do bloco	botões do painel (cada clique passa para a próxima opção)

O painel mostra, para cada algoritmo, o **custo** do caminho (em ticks), os **nós expandidos**, o **tempo** de execução e se o agente **chega vivo**. Com "Explorados" ligado, as células que o algoritmo selecionado examinou ficam coloridas, e isso mostra o esforço de cada busca.

Como estudar

1. **Rode o jogo primeiro.** Troque de algoritmo, coloque bombas e anime. Ver o comportamento antes da teoria ajuda muito.
2. **Leia os capítulos 2 a 4** (problema, busca e modelagem). Eles explicam os dois exemplos do script `exemplos/passo_a_passo.py` linha por linha; rode o script junto.
3. **Leia o código com o capítulo 5 aberto.** Os arquivos centrais são curtos: `ia/busca.py` e `ia/problema.py` têm cerca de 100 linhas cada.
4. **Faça os experimentos do capítulo 8.** Mudar algo e prever o resultado é o melhor teste de entendimento.

2. O jogo e o problema

O Bomberman em 30 segundos

No jogo	Na interface	No código (jogo/mapa.py)
parede indestrutível	cinza	`PAREDE = "#"`
bloco destrutível	marrom com tijolos	`BLOCO = "+"`
piso livre	verde	`LIVRE = "."`
bomba	círculo preto com o número de ticks até explodir	`Bomba(x, y, tempo, alcance)`
saída	quadrado azul	`mapa.objetivo`

As paredes formam a borda e "pilares" nas posições em que `x` e `y` são pares: é o padrão clássico do jogo, criado pela função `gerar_mapa`. As demais células recebem um bloco com a probabilidade escolhida (20%, 40% ou 60%). Como no jogo original, o canto inicial sempre tem um "L" de células livres para o agente conseguir fugir da primeira bomba.

Quando uma bomba explode, o fogo se espalha em cruz até o **alcance** dela e:

- para nas paredes;
- destrói o primeiro bloco que encontra e para nele;
- mata quem estiver no caminho, inclusive quem plantou a bomba;
- faz qualquer outra bomba atingida explodir no mesmo instante (**reação em cadeia**);
- permanece por 2 ticks.

O que o agente pode fazer

- andar uma célula para cima, baixo, esquerda ou direita (1 tick);
- **quebrar um bloco** para atravessá-lo: fica parado 4 ticks quebrando e depois entra (1 + 4 = 5 ticks);
- **esperar** parado (1 tick).

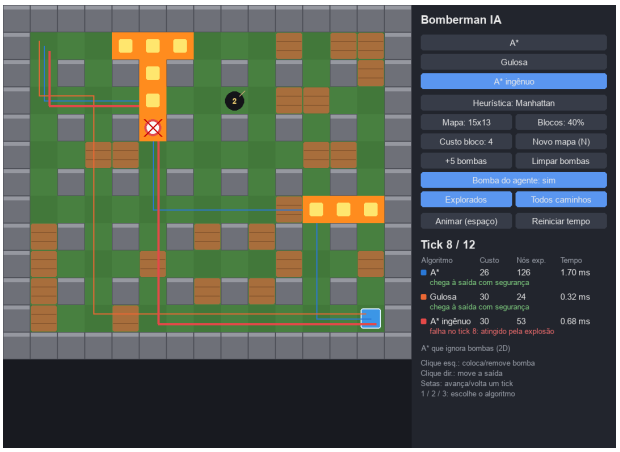
Com o botão "Bomba do agente" ligado, ele começa em cima de uma bomba que ele mesmo plantou (explode no tick 3, alcance 2), então a primeira coisa a fazer é sair de perto dela.

Por que não é um labirinto comum

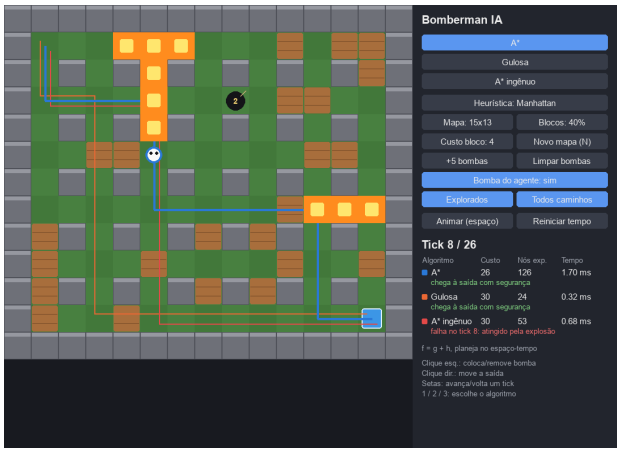
1. Nem todo passo custa o mesmo. Atravessar um bloco custa 5 ticks. Às vezes vale mais a pena dar a volta: um caminho com mais células pode chegar mais cedo.

2. O mapa muda com o tempo. Uma célula segura agora pode estar em chamas daqui a 3 ticks. E um bloco que fecha a passagem agora pode ser destruído por uma explosão, abrindo um atalho. Um caminho que parece perfeito no tick 0 pode levar o agente direto para o fogo.

As duas imagens abaixo são do **mesmo mapa, no mesmo tick 8**:



A ingênuo (ignora o tempo): entrou no corredor na hora errada e foi atingido (X vermelho).*



A no espaço-tempo: planejou contando com as explosões e segue em segurança.*

> **Ideia-chave.** a solução é planejar no **espaço-tempo**. O agente não pergunta só "onde posso ir?", mas "onde posso estar **em cada tick**?". O capítulo 4 mostra como isso entra na busca.

3. Busca do zero

Antes do Bomberman, os conceitos gerais. Tudo aqui vale para qualquer problema de busca.

Os ingredientes de um problema de busca

Conceito	O que é	Exemplo num mapa sem bombas
Estado	uma "foto" com tudo o que importa para decidir	a posição do agente, (3, 1)
Estado inicial	onde a busca começa	(1, 1)
Ações	o que dá para fazer em um estado	andar para cima, baixo, esquerda, direita
Custo	quanto cada ação custa	1 tick (ou 5, se for um bloco)
Objetivo	o teste que diz "cheguei"	a posição é a saída

Com esses ingredientes, o mapa vira um **grafo**: cada estado é um vértice e cada ação é uma aresta com um custo. Achar o melhor caminho no jogo é achar o caminho de menor custo nesse grafo.

Como uma busca funciona

Toda busca deste projeto segue o mesmo ciclo. Ela mantém uma **fronteira**: a lista de estados já descobertos que ainda não foram examinados. Começa só com o estado inicial e repete:

1. **Escolhe** um nó da fronteira (o de menor nota f).
2. Se ele é o objetivo, **terminou**: devolve o caminho.
3. Senão, **expande** o nó: gera os vizinhos (os estados alcançáveis com uma ação) e coloca na fronteira os que ainda não foram vistos.
4. Marca o nó como **fechado** (já examinado) e volta ao passo 1.

O que muda de um algoritmo para outro é **só o passo 1**: o critério para escolher o próximo nó. Esse critério é a função de avaliação **f(n)**, montada a partir de dois números:

Símbolo	Nome	Significado	No projeto
g(n)	custo real	quanto já foi gasto do início até n	o tick em que o agente chega em n
h(n)	heurística	uma *estimativa* de quanto falta de n até a saída	distância de Manhattan até a saída

Busca Gulosa: f(n) = h(n)

A Gulosa escolhe sempre o nó que **parece** mais perto da saída. Ela não olha para quanto já gastou. Por isso é rápida (vai direto ao ponto), mas pode escolher um caminho caro: se o vizinho mais perto da saída for um bloco que custa 5 ticks, ela vai nele mesmo assim.

A*: f(n) = g(n) + h(n)

O A* soma o que já gastou com o que estima que falta. f(n) é então uma estimativa do **custo total** de um caminho que passa por n. O A* sempre expande o nó com a menor estimativa de custo total, o que o faz contornar um bloco caro se a volta for mais barata.

Heurística admissível: a condição para o A* ser ótimo

Uma heurística é **admissível** se nunca superestima o custo real até a saída. Com uma heurística admissível, o A* **garante** encontrar o caminho de menor custo. A intuição:

- Se h nunca exagera, f(n) = g(n) + h(n) nunca passa do custo real do melhor caminho que passa por n.
- O A* só para quando retira a saída da fronteira. Nesse momento, a saída tem o menor f entre todos os nós abertos.
- Se existisse um caminho mais barato, algum nó dele estaria na fronteira com f menor, e teria sido escolhido antes.

A distância de Manhattan, $|x - x_s| + |y - y_s|$, é o número mínimo de passos numa grade sem diagonais. O agente não chega na saída em menos passos que isso, e cada passo custa pelo menos 1 tick. Então ela nunca exagera: é admissível.

> **Ideia-chave.** os dois algoritmos são o **mesmo código** com uma única diferença: a função f. Gulosa = só h (rápida, sem garantia). A* = g + h (mais trabalho, caminho ótimo garantido).

Exemplo 1: Gulosa e A* no mesmo mapa

Este é o primeiro exemplo do script `exemplos/passo_a_passo.py`. O bloco no meio do corredor de cima custa 1 + 8 ticks (`custo_bloco=8`). O agente começa em S (1, 1) e a saída E fica em (7, 1).

```
#########
#S..+..E#
#.#.#.#.#
#.......#
#########

colunas x = 0..8 (esquerda → direita), linhas y = 0..4 (cima → baixo)
```

Há duas rotas: **por cima**, atravessando o bloco (6 passos, sendo um deles o bloco: 2 + 9 + 3 = **14 ticks**), e **por baixo**, contornando (10 passos livres = **10 ticks**). A melhor é por baixo.

Gulosa: custo 14 ticks, 7 nós expandidos

#	estado (x, y, t)	g	h	f = g + h
1	(1, 1, 0)	0	6	6
2	(2, 1, 1)	1	5	6
3	(3, 1, 2)	2	4	6
4	(4, 1, 11)	11	3	14
5	(5, 1, 12)	12	2	14
6	(6, 1, 13)	13	1	14
7	(7, 1, 14) — saída!	14	0	14

Linha a linha: a Gulosa só olha a coluna **h**. Em (3, 1), os vizinhos são o bloco (4, 1), com h = 3, e (3, 2), com h = 5. Ela escolhe o bloco porque h é menor, sem perceber que g pula de 2 para 11 (linha destacada). Depois segue reto até a saída: **14 ticks**.

A*: custo 10 ticks, 12 nós expandidos

#	estado (x, y, t)	g	h	f = g + h
1	(1, 1, 0)	0	6	6
2	(2, 1, 1)	1	5	6
3	(3, 1, 2)	2	4	6
4	(3, 2, 3)	3	5	8
5	(1, 2, 1)	1	7	8
6	(3, 3, 4)	4	6	10
7	(4, 3, 5)	5	5	10
8	(5, 3, 6)	6	4	10
9	(5, 2, 7)	7	3	10
10	(5, 1, 8)	8	2	10
11	(6, 1, 9)	9	1	10
12	(7, 1, 10) — saída!	10	0	10

O começo é igual. A diferença aparece em (3, 1): o bloco entra na fronteira com **f = 11 + 3 = 14**, e (3, 2) com **f = 3 + 5 = 8**. O A* prefere o menor f e desce. O nó (1, 2), gerado lá no início com f = 8, também é expandido. Depois o A* segue pela rota de baixo, com f = 10 o tempo todo, e chega à saída em **10 ticks**. O bloco, com f = 14, nunca chega a ser expandido.

> **Resumo do exemplo.** a Gulosa expandiu menos nós (7 contra 12), mas achou um caminho 40% mais lento. O A* trabalhou um pouco mais e garantiu o ótimo. Esse é exatamente o trade-off que os resultados do capítulo 6 medem em mapas grandes.

Repare também que, no A*, o valor de f nunca diminui ao longo de um caminho (6, 8, 10). Isso acontece porque a heurística é **consistente**, uma propriedade um pouco mais forte que a

admissibilidade, explicada no capítulo 4.

4. Como o Bomberman vira um problema de busca

O capítulo 3 usou estados só com posição. Para lidar com as bombas, o projeto coloca o **tempo** dentro do estado. Tudo isto está em `ia/problema.py`.

Os ingredientes, agora com tempo

Ingrediente	Definição no projeto
Estado	`(x, y, t)`: coluna, linha e o tick atual
Estado inicial	`(1, 1, 0)`
Ações	cima, baixo, esquerda, direita e **esperar**
Custo	a **duração** da ação: 1 tick numa célula livre, 1 + `custo_bloco` num bloco, 1 para esperar
Objetivo	`(x, y)` é a saída, em qualquer tick

Como o custo de cada ação é a duração dela, o custo acumulado é o próprio relógio: **g(n) = t**. Minimizar o custo é minimizar o tempo até a saída.

Restrições: os estados em que o agente morreria não existem

O método `sucessores(estado)` só gera uma ação se ela for possível e segura. Uma ação é descartada se:

1. o destino é parede ou fica fora do mapa;
2. o destino tem uma bomba que ainda não explodiu (bombas bloqueiam a passagem, como no jogo);
3. o destino estará **em chamas no tick de chegada**;
4. ao quebrar um bloco, a célula onde o agente fica parado pega fogo durante a espera;
5. ao esperar, a célula atual estará em chamas no próximo tick.

Este é o código real do método:

```
def sucessores(self, estado: Estado) -> Iterator[Transicao]:
    x, y, t = estado
    cron = self.cronograma

    for acao, (dx, dy) in DIRECOES.items():
        nx, ny = x + dx, y + dy
        destino = (nx, ny)
        if not self.mapa.dentro(nx, ny) or self.mapa.tipo(nx, ny) == PAREDE:
            continue
        if cron.tem_bomba(destino, t + 1):
            continue

        duracao = 1 + self.custo_bloco if cron.tem_bloco(self.mapa, destino, t + 1) else 1
        chegada = t + duracao
        # enquanto quebra o bloco, o agente fica parado na célula atual
        if duracao > 1 and not self._seguro((x, y), t + 1, chegada - 1):
            continue
        if cron.perigoso(destino, chegada):
            continue
        yield Transicao(acao, (nx, ny, chegada), duracao)

    # esperar só faz sentido enquanto o mapa ainda vai mudar
    if t < cron.horizonte and not cron.perigoso((x, y), t + 1):
        yield Transicao("esperar", (x, y, t + 1), 1)
```

> **Ideia-chave.** a segurança vem da **modelagem**, não do algoritmo. Como um estado em que o agente morreria nunca é gerado, **qualquer** busca sobre esse modelo, inclusive a Gulosa, só devolve caminhos seguros. O que muda entre os algoritmos é a qualidade (o tempo de chegada).

O cronograma de explosões

Para responder "esta célula está em chamas no tick 12?", seria possível simular as bombas a cada passo da busca, mas isso seria lento. A observação que resolve o problema: **o tempo de cada bomba é conhecido, então o futuro do mapa é determinístico**. O arquivo `jogo/explosoes.py` calcula tudo uma vez, antes da busca, e monta um **cronograma**:

Campo do Cronograma	Conteúdo	Usado para
`perigo`	célula → conjunto de ticks com fogo	restrições 3, 4 e 5
`bomba_ate`	célula → tick em que a bomba explode	restrição 2
`bloco_destruido`	célula → tick em que o bloco some	bloco destruído por explosão vira piso (custo 1)
`horizonte`	último tick em que algo muda	limitar o espaço de estados (próxima seção)

A função `calcular_cronograma` processa as bombas em ordem de explosão, usando uma fila de prioridade:

1. retira a bomba que explode mais cedo;
2. espalha o fogo em cruz: para na parede; no primeiro bloco ainda de pé, entra, registra a destruição e para;

3. se o fogo atinge outra bomba que explodiria mais tarde, **antecipa** a explosão dela para o tick atual e a coloca de volta na fila (reação em cadeia);
4. marca as células atingidas como perigosas por 2 ticks.

Depois disso, cada consulta durante a busca é uma busca em dicionário: custo O(1).

O horizonte: por que o espaço de estados não é infinito

Com o tempo dentro do estado, surge um problema: o agente poderia esperar para sempre, gerando (1, 1, 0), (1, 1, 1), (1, 1, 2), ... sem fim. A solução é o **horizonte**: o tick a partir do qual nenhuma bomba explode e nenhum fogo existe. Depois dele, o mapa nunca mais muda, então:

- estar em (x, y) no tick 30 ou no tick 50 é a mesma situação, se ambos são depois do horizonte. O método `chave()` junta esses estados em `(x, y, horizonte)`;
- a ação **esperar** só é gerada antes do horizonte, porque depois dele esperar nunca ajuda.

Assim, existem no máximo *células livres* × *(horizonte + 1)* estados. Sem bombas, o horizonte é 0 e o problema vira um pathfinding 2D comum, como o do capítulo 3.

As heurísticas e por que elas funcionam

Heurística	Fórmula	Comentário
Manhattan (padrão)	`|dx| + |dy|`	mínimo de passos numa grade de 4 direções
Euclidiana	`√(dx² + dy²)`	linha reta; sempre ≤ Manhattan, então menos informada
Nula	`0`	transforma o A* no algoritmo de Dijkstra; serve de referência

Uma heurística é **consistente** quando, para toda ação de n para n' com custo c, vale `h(n) ≤ c + h(n')`. Em palavras: dar um passo nunca reduz a estimativa mais do que o passo custou. A Manhattan é consistente aqui porque:

- cada movimento muda x **ou** y em 1, então muda h em no máximo 1, e custa pelo menos 1 tick;
- esperar não muda a posição (h fica igual) e custa 1;
- os blocos só aumentam o custo real, então nunca quebram a desigualdade.

Toda heurística consistente é admissível, então o A* é ótimo. A consistência também garante que, quando o A* expande um estado, ele já chegou nesse estado pelo caminho mais barato. Por isso f nunca diminui ao longo de um caminho, como visto no exemplo 1.

Exemplo 2: esperar o fogo passar

O segundo exemplo do script: um corredor com uma bomba em (3, 1) que explode no tick 2, com alcance 1. O fogo cobre as células (2, 1), (3, 1) e (4, 1) nos ticks 2 e 3.

```
#######
#S.B.E#
#######

B = bomba (tempo 2, alcance 1)
```

A* ingênuo (ignora a bomba): planeja 4 ticks e morre

#	estado (x, y, t)	g	h	f = g + h
1	(1, 1, 0)	0	4	4
2	(2, 1, 1)	1	3	4
3	(3, 1, 2)	2	2	4
4	(4, 1, 3)	3	1	4
5	(5, 1, 4) — saída!	4	0	4

O A* ingênuo não vê a bomba e anda reto. No tick 2 ele estaria em (3, 1), exatamente quando a bomba explode (linha destacada). Na simulação real, o agente morre.

A* no espaço-tempo: 7 ticks e sobrevive

#	estado (x, y, t)	g	h	f = g + h
1	(1, 1, 0)	0	4	4
2	(2, 1, 1)	1	3	4
3	(1, 1, 1)	1	4	5
4	(1, 1, 2)	2	4	6
5	(1, 1, 3)	3	4	7
6	(2, 1, 4)	4	3	7
7	(3, 1, 5)	5	2	7
8	(4, 1, 6)	6	1	7
9	(5, 1, 7) — saída!	7	0	7

Ações escolhidas: **direita** → **esquerda** → **esperar** → **direita** → **direita** → **direita** → **direita**.

O A* tenta avançar, mas a partir de (2, 1) no tick 1 não pode seguir em frente nem ficar parado, porque (3, 1) e (2, 1) estarão em chamas no tick 2. O único sucessor seguro é voltar para (1, 1). Ele espera até o fogo sumir (tick 4) e só então atravessa o corredor, chegando no tick 7.

> **Detalhe interessante.** o plano "esperar, esperar, esperar, direita..." também custa 7. Existem vários caminhos ótimos, e o A* devolve o primeiro que encontrou. A garantia do A* é sobre o **custo**, não sobre qual dos caminhos ótimos ele escolhe.

O A* ingênuo

O terceiro planejador é o mesmo A*, criado com `considerar_tempo=False`. Ele recebe um cronograma vazio: não vê bombas, fogo nem blocos destruídos. Depois, o plano dele é executado contra o cronograma **real** pela função `verificar_plano` (em `jogo/simulacao.py`), que diz se o agente sobreviveria e em que tick morreria. Ele existe para provar, com números, que colocar o tempo no estado é necessário.

5. Tour pelo código

Organização das pastas

```
bomberman-ia/
├── main.py               abre a interface
├── jogo/
│   ├── mapa.py           grade, geração aleatória, bombas
│   ├── explosoes.py      cronograma de explosões (fogo, cadeia, blocos)
│   ├── simulacao.py      plano → posição por tick; verifica se sobrevive
│   └── interface.py      interface Pygame
├── ia/
│   ├── problema.py       estados, ações, custos e restrições
│   ├── heuristicas.py    Manhattan, Euclidiana, Nula
│   └── busca.py          A* e Busca Gulosa
├── benchmark/
│   ├── executar.py       4 cenários de teste, CSVs e tabelas
│   └── graficos.py       gráficos dos resultados
├── exemplos/
│   └── passo_a_passo.py  rastreio nó a nó (exemplos 1 e 2)
├── testes/
│   └── test_busca.py     testes automatizados
├── resultados/           saída do benchmark
└── docs/                 este guia e as imagens
```

A divisão segue uma regra: `ia/` **não sabe nada de interface**, e `jogo/` não sabe nada de busca (exceto a interface, que junta tudo). Por isso os mesmos algoritmos rodam na interface, no benchmark e nos testes sem nenhuma mudança.

O que acontece quando você clica no mapa

Seguir um clique do começo ao fim é a melhor forma de entender como as peças se conectam:

1. `App.clique_no_mapa` (interface.py) adiciona uma `Bomba` na lista e chama `replanejar()`.
2. `replanejar` monta o mapa com as bombas atuais e calcula o cronograma com `calcular_cronograma` (explosoes.py).
3. Cria dois problemas: `ProblemaBomberman(mapa)`, com tempo, e `ProblemaBomberman(mapa, considerar_tempo=False)`, o ingênuo.
4. Roda `a_estrela` e `busca_gulosa` no primeiro problema e `a_estrela` no ingênuo (busca.py), com a heurística escolhida.
5. Para cada resultado, `expandir_plano` transforma o caminho numa posição por tick, e `verificar_plano` confere contra o cronograma real se o agente sobrevive (simulacao.py).
6. A cada quadro (60 por segundo), `desenhar()` pinta o mapa no tick atual: blocos ainda de pé, fogo, bombas com contagem regressiva, células exploradas, caminhos e o agente.

ia/busca.py: o coração do projeto

Os dois algoritmos são duas linhas que chamam a mesma função. A única diferença é a função f e se estados podem ser reabertos:

```
def a_estrela(problema: ProblemaBomberman, h: Heuristica) -> Resultado:
    return _melhor_primeiro(problema, h, lambda g, hn: g + hn, "A*", reabrir=True)


def busca_gulosa(problema: ProblemaBomberman, h: Heuristica) -> Resultado:
    return _melhor_primeiro(problema, h, lambda g, hn: hn, "Gulosa", reabrir=False)
```

E esta é a função de busca completa:

```
def _melhor_primeiro(
    problema: ProblemaBomberman,
    h: Heuristica,
    f: Callable[[float, float], float],
    nome: str,
    reabrir: bool,
) -> Resultado:
    """Busca em grafo de melhor-primeiro.

    `reabrir=True` (A*): um estado já visto volta para a fronteira se for alcançado por
    um caminho mais barato. `reabrir=False` (Gulosa): cada estado entra uma vez só.
    """
    inicio_relogio = time.perf_counter()
    objetivo = problema.objetivo
    desempate = itertools.count()  # FIFO entre nós com o mesmo f e h

    s0 = problema.estado_inicial()
    k0 = problema.chave(s0)
    h0 = h(s0[:2], objetivo)
    fronteira = [(f(0, h0), h0, next(desempate), 0, s0)]
    melhor_g: Dict[Estado, float] = {k0: 0}
    pai: Dict[Estado, Tuple[Optional[Estado], Optional[str], Estado]] = {k0: (None, None, s0)}
    fechados = set()
    resultado = Resultado(nome, sucesso=False, nos_gerados=1)

    while fronteira:
        _, _, _, g, estado = heapq.heappop(fronteira)
        chave = problema.chave(estado)
        if chave in fechados or g > melhor_g[chave]:
            continue  # entrada obsoleta
        fechados.add(chave)
        resultado.nos_expandidos += 1
        resultado.ordem_expansao.append(estado[:2])

        if problema.eh_objetivo(estado):
            resultado.sucesso = True
            resultado.custo = g
            resultado.caminho, resultado.acoes = _reconstruir(pai, chave)
            break

        for transicao in problema.sucessores(estado):
            novo = transicao.estado
            k = problema.chave(novo)
            novo_g = g + transicao.custo
            if k in fechados:
                continue
            ja_visto = k in melhor_g
            if ja_visto and (not reabrir or novo_g >= melhor_g[k]):
                continue
            melhor_g[k] = novo_g
            pai[k] = (chave, transicao.acao, novo)
            hn = h(novo[:2], objetivo)
            heapq.heappush(fronteira, (f(novo_g, hn), hn, next(desempate), novo_g, novo))
            resultado.nos_gerados += 1

    resultado.tempo_ms = (time.perf_counter() - inicio_relogio) * 1000
    return resultado
```

Pontos para entender nesse código

- **A fronteira é um heap** (`heapq`): retirar o menor elemento custa O(log n). Cada entrada é a tupla `(f, h, contador, g, estado)`. O heap compara as tuplas da esquerda para a direita: primeiro f; empate em f, prefere o menor h (o nó mais perto da saída); empate em tudo, o `contador` garante a ordem de chegada (FIFO).
- **Remoção preguiçosa.** O heap não permite mudar a prioridade de um item. Quando o A* acha um caminho mais barato para um estado, ele insere uma entrada nova e deixa a velha lá. Ao retirar, o teste `g > melhor_g[chave]` reconhece a entrada velha e a pula.
- **O objetivo é testado ao retirar, não ao gerar.** Um nó gerado cedo pode ter um caminho melhor que ainda não foi descoberto. Testar só na retirada é o que garante a otimalidade do A*.
- `reabrir`**.** No A*, um estado já visto volta para a fronteira se for alcançado com g menor. Na Gulosa, cada estado entra uma vez só: ela não se importa com g, então não teria por que reabrir.
- `pai` guarda, para cada estado, de onde ele veio e com qual ação. No final, `_reconstruir` segue os pais de trás para frente e inverte a lista.
- `ordem_expansao` não é usado pela busca. Ele só registra as células expandidas para a interface pintar os "explorados".

Os outros arquivos, em resumo

Arquivo	Principais funções	Para que servem
`jogo/mapa.py`	`gerar_mapa`, `bombas_aleatorias`	criam mapas no padrão do jogo e sorteiam bombas (tempo de 2 a 10, alcance de 1 a 3)
`jogo/explosoes.py`	`calcular_cronograma`	simula todas as explosões e devolve o `Cronograma`
`jogo/simulacao.py`	`expandir_plano`, `verificar_plano`	transformam o caminho em posições por tick e conferem se o agente sobrevive
`ia/heuristicas.py`	`manhattan`, `euclidiana`, `nula`	as três heurísticas e o dicionário `HEURISTICAS`
`ia/problema.py`	`ProblemaBomberman`	estado inicial, objetivo, sucessores e chave do estado
`jogo/interface.py`	`App`	janela, botões, desenho e animação
`benchmark/executar.py`	`cenario_*`, `resumir`	rodam os cenários em 30 mapas cada e calculam médias

Os testes automatizados

O arquivo `testes/test_busca.py` verifica que o código está correto, não apenas que roda:

Teste	O que garante
`test_a_estrela_encontra_o_mesmo_custo_que_dijkstra`	em 50 mapas aleatórios, com e sem bombas, o A* (Manhattan e Euclidiana) acha o mesmo custo que o Dijkstra, que é sabidamente ótimo

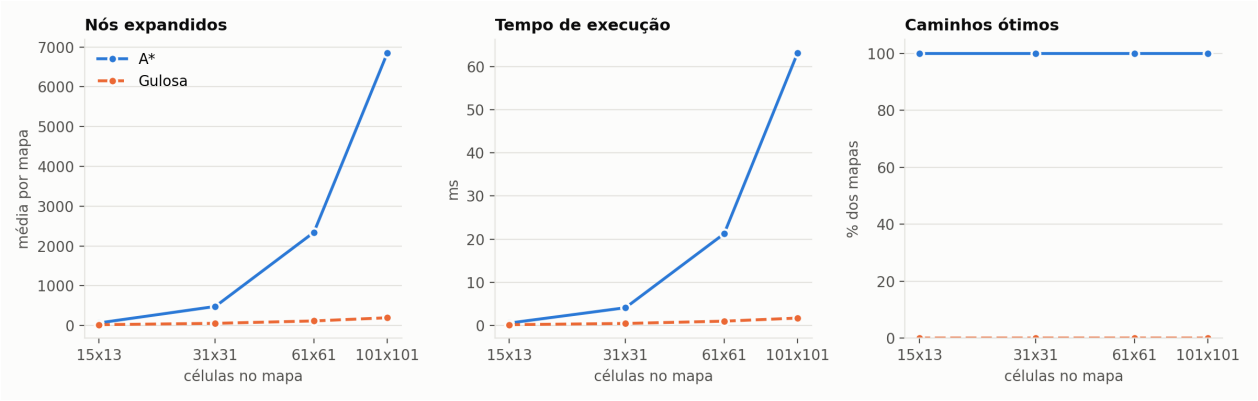
Teste	O que garante
`test_gulosa_nunca_e_melhor_que_o_otimo`	a Gulosa acha caminho sempre que existe um, e nunca mais barato que o A*
`test_planos_com_tempo_sobrevivem_as_explosoes`	todo plano do A* e da Gulosa, executado contra o cronograma real, sobrevive
`test_custo_do_bloco_e_contabilizado`	atravessar um bloco custa 1 + custo_bloco
`test_agente_espera_a_explosao_passar`	o ingênuo morre no corredor e o A* espera e sobrevive (exemplo 2)
`test_agente_foge_da_propria_bomba`	o agente consegue fugir da bomba que plantou
`test_explosao_para_na_parede_e_no_bloco`	o fogo para nas paredes e no primeiro bloco, que é destruído
`test_reacao_em_cadeia`	uma bomba atingida explode no mesmo tick da que a atingiu

6. Resultados

O benchmark (`benchmark/executar.py`) roda cada configuração em **30 mapas aleatórios** (sementes fixas, então os resultados são reproduzíveis) e tira a média. Os números completos estão em `resultados/resumo.md` e nos CSVs. O tempo de execução varia um pouco de máquina para máquina; os nós expandidos e os custos são sempre os mesmos.

As métricas são: **nós expandidos** (esforço da busca), **tempo** (ms), **custo** (ticks até a saída) e **% ótimo** (em quantos mapas o custo foi igual ao do A*).

Cenário 1: tamanho do mapa

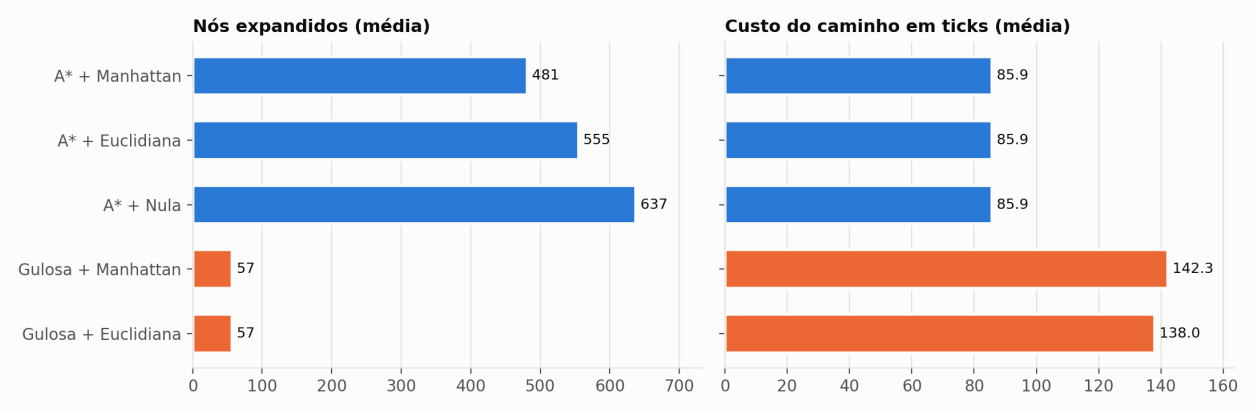


Nós expandidos, tempo e % de caminhos ótimos por tamanho de mapa (40% de blocos, sem bombas).

Mapa	Nós A*	Nós Gulosa	Custo A*	Custo Gulosa	Gulosa mais lenta em
15x13	68	23	33.9	52.4	55%
31x31	481	57	85.9	142.3	66%
61x61	2344	117	176.0	293.1	67%
101x101	6855	197	289.9	513.3	77%

- **O esforço do A* cresce com a área do mapa**: ele precisa comparar muitas alternativas. A Gulosa cresce só com a distância até a saída.
- **A Gulosa expande quase só o próprio caminho.** Em mapas sem bombas, sempre existe um vizinho mais perto da saída (para a direita ou para baixo), porque blocos são atravessáveis. Ela nunca volta atrás e vai em "escada" até a saída, atravessando todo bloco no caminho.
- **Por isso a Gulosa não acertou o ótimo em nenhum mapa**: a heurística não enxerga o custo dos blocos, e a Gulosa só olha a heurística.

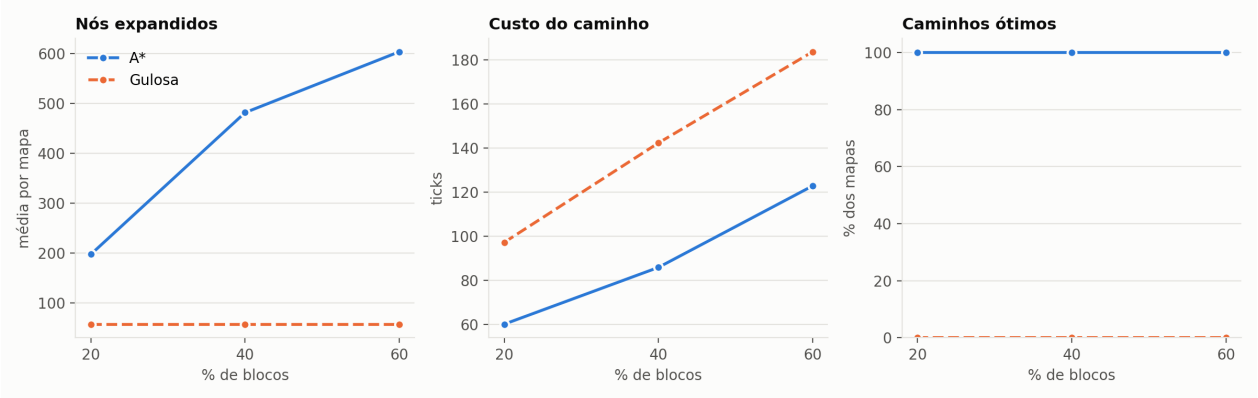
Cenário 2: heurísticas



Nós expandidos e custo médio por combinação de algoritmo e heurística (mapas 31x31, 40% de blocos).

- **As três heurísticas levam o A* ao mesmo custo**, porque todas são admissíveis.
- **Quanto mais informada a heurística, menos nós o A* expande**: Manhattan (481) < Euclidiana (555) < Nula/Dijkstra (637). A Manhattan economiza cerca de 25% do trabalho em relação ao Dijkstra sem perder a otimalidade.
- Na Gulosa, a troca de heurística muda pouco: as duas levam a caminhos ruins.

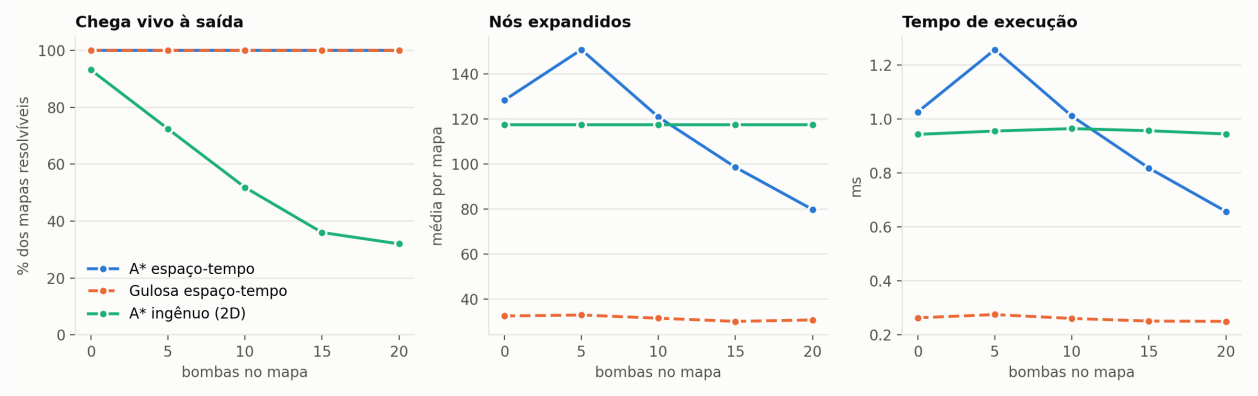
Cenário 3: densidade de blocos



Efeito da porcentagem de blocos (mapas 31x31).

- Mais blocos tornam os caminhos mais caros para os dois algoritmos.
- O A* expande mais nós com mais blocos: há mais alternativas de custo parecido para comparar.
- A Gulosa expande sempre o mesmo número de nós (ela segue a mesma "escada"), mas o custo dela quase dobra de 20% para 60% de blocos, porque atravessa cada vez mais blocos.

Cenário 4: fuga das explosões



Planejamento no espaço-tempo x A ingênuo com 0 a 20 bombas (mapas 21x17, com a bomba do agente).*

Bombas	Mapas com saída segura	Sobrevive: A*	Sobrevive: Gulosa	Sobrevive: ingênuo
0	97%	100%	100%	93%
5	97%	100%	100%	72%
10	90%	100%	100%	52%
15	83%	100%	100%	36%
20	83%	100%	100%	32%

- **Este é o resultado principal do projeto.** O A* ingênuo morre cada vez mais conforme o número de bombas aumenta. Os dois planejadores no espaço-tempo sobrevivem em 100% dos mapas que têm saída segura.
- Mesmo sem bombas extras, o ingênuo morre em alguns mapas: são os casos em que o caminho dele começa quebrando um bloco perto do início, e ele fica parado dentro do alcance da própria bomba.
- A Gulosa também sobrevive em 100%: confirma que a segurança vem do modelo, não do algoritmo.
- **Com mais bombas, o A* chega mais rápido** (o custo médio cai de cerca de 41 para 35 ticks). As explosões destroem blocos, e o A* aproveita: espera uma bomba abrir a passagem em vez de gastar 5 ticks quebrando o bloco.
- Alguns mapas não têm nenhuma saída segura (o agente fica encurralado). Nesses casos o A* esgota a busca e informa que não há caminho, o que é o comportamento correto.

7. Preparação para a apresentação

Roteiro sugerido para a demonstração

1. **Mapa sem bombas** (desligue "Bomba do agente"). Mostre o A* e a Gulosa com "Explorados" ligado: a Gulosa explora pouco e vai reto; o A* explora mais e contorna os blocos. Compare custo e nós no painel.
2. **Custo do bloco em 8.** O A* passa a contornar ainda mais; a Gulosa continua atravessando.
3. **Heurística Nula.** O custo do A* não muda, mas os nós expandidos aumentam: é o Dijkstra.
4. **Ligue a bomba do agente e clique em "+5 bombas"** algumas vezes. Selecione o A* ingênuo e anime até ele morrer. Depois volte para o tick 0, selecione o A* e anime: ele espera e desvia.
5. **Use as setas** para avançar tick a tick num momento em que o A* espera parado, e explique o estado (x, y, t).
6. **Coloque uma bomba com o mouse** no caminho do A*: ele replaneja na hora.

Perguntas prováveis e respostas

Por que a distância de Manhattan é admissível?

Porque é o número mínimo de passos numa grade sem diagonais, e cada passo custa pelo menos 1 tick. Blocos e esperas só aumentam o custo real, então ela nunca superestima.

Por que o A* é ótimo e a Gulosa não?

O A* ordena por g + h, uma estimativa do custo total que nunca exagera, e só para quando a saída tem o menor f da fronteira. A Gulosa ignora g e pode escolher um vizinho caro só porque parece perto. O exemplo 1 do capítulo 3 mostra isso.

Por que colocar o tempo no estado?

Porque a mesma célula pode ser segura num tick e mortal no seguinte. Sem o tempo, a busca não consegue representar "esperar o fogo passar". O cenário 4 mostra o ingênuo morrendo em até 68% dos mapas.

Com o tempo no estado, o espaço de estados não fica infinito?

Não, por causa do horizonte. Depois da última explosão o mapa é estático, então os estados com t ≥ horizonte são agrupados e a ação esperar deixa de existir.

A Gulosa pode levar o agente à morte?

Não. As restrições de fogo estão na geração de sucessores, então nenhum algoritmo sobre esse modelo gera um estado mortal.

Por que só o A* reabre estados?

Porque o A* precisa garantir o menor g de cada estado. A Gulosa ignora g, então reabrir não mudaria nada para ela.

Qual a complexidade?

No pior caso O(V log V), sendo V o número de estados (células livres × (horizonte + 1)), por causa do heap. Na prática a heurística reduz muito os nós expandidos (cenário 2).

Como você sabe que o A* está correto?

Os testes comparam o A* com o Dijkstra em 50 mapas aleatórios, com e sem bombas, e conferem que os custos são iguais. Também simulam todo plano contra o cronograma real para garantir que o agente sobrevive.

Por que quebrar um bloco custa 1 + 4?

É uma abstração do tempo de plantar uma bomba, se afastar e esperar. O valor é um parâmetro (2, 4 ou 8 na interface).

O que acontece se não existir caminho seguro?

A busca esgota a fronteira e devolve falha. A interface mostra "sem caminho seguro".

Limitações

- **Quebrar bloco é uma macro-ação** de custo fixo: o agente não planta de verdade uma bomba para isso. Fazer isso exigiria guardar o mapa inteiro no estado.
- **Não há inimigos se movendo.** Os únicos perigos são as bombas, cujo tempo é conhecido de antemão.
- **O fogo é verificado tick a tick:** o modelo não trata a travessia "entre" dois ticks.
- **A heurística ignora blocos e bombas**, então em mapas com muitos blocos ela subestima bastante o custo, e o A* explora mais do que precisaria.

Possíveis melhorias

- **Heurística mais informada:** pré-calcular, com um Dijkstra a partir da saída no mapa sem bombas, o custo real considerando os blocos. Continua admissível (as bombas só pioram) e reduziria os nós expandidos.
- **Inimigos:** replanejar a cada tick, ou usar algoritmos incrementais como o D* Lite.
- **Plantar bombas como ação real**, com o mapa no estado e técnicas para controlar a explosão de estados.
- **Weighted A*** ($f = g + w \cdot h$), um meio-termo entre A* e Gulosa com garantia de custo no máximo w vezes o ótimo.

8. Experimentos para fazer sozinho

Para cada experimento, tente **prever o resultado antes de rodar**. Se errar, releia a seção indicada.

Experimento	O que observar	Seção
Na interface, alterne entre Manhattan, Euclidiana e Nula com o A* selecionado	o custo não muda; os nós expandidos e a área colorida aumentam	3 e 6 (cenário 2)
Mude o custo do bloco de 2 para 8	o A* contorna mais; a rota da Gulosa não muda	3, exemplo 1
No exemplo 1, mude `custo_bloco=8` para `custo_bloco=2`	a rota pelo bloco passa a custar 2 + 3 + 3 = 8 < 10, e o A* também atravessa o bloco	3
No exemplo 2, mude o tempo da bomba de 2 para 6	o ingênuo passa antes da explosão e sobrevive	4
Em `ia/busca.py`, troque `reabrir=True` por False no A* e rode os testes	nos mapas dos testes o A* continua acertando, mas isso não é garantido em geral: imagine um estado descoberto primeiro por um caminho mais caro	4
Crie uma heurística inadmissível, como `2 * manhattan`, e rode os testes	o teste de otimalidade passa a falhar: o A* deixa de garantir o ótimo	3
Mude o alcance da bomba do agente (em `interface.py`) de 2 para 3	mais mapas sem saída segura; o ingênuo morre mais	6 (cenário 4)

Referências

- HART, P. E.; NILSSON, N. J.; RAPHAEL, B. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. *IEEE Transactions on Systems Science and Cybernetics*, v. 4, n. 2, p. 100–107, 1968.
- RUSSELL, S.; NORVIG, P. *Inteligência Artificial: uma abordagem moderna*. 4. ed. Capítulo 3 (busca informada).
- RESNICK, C. et al. Pommerman: A Multi-Agent Playground. *arXiv:1809.07124*, 2018.
- SILVER, D. Cooperative Pathfinding. In: *AIIDE*, 2005.